UNIVERSIDADE FEDERAL DO PARÁ
CAMPUS UNIVERSITÁRIO DE TUCURUÍ
FACULDADE DE ENGENHARIA DE COMPUTAÇÃO

Simulador Interativo de Autômatos Finitos e Gramáticas Regulares com Conversão AFND → AFD e Minimização

DOCENTE: OTÁVIO NOURA TEIXEIRA

DISCIPLINA: TEORIA DE COMPUTAÇÃO I

DISCENTES:

ALEXANDRE MELGAÇO CHAVES SILVA

FABRICIO CRISTIAN FORMENTO DE SOUSA

TADEU HENRIQUE DA CRUZ FERNANDES

TUCURUÍ - PA

2026

1. INTRODUÇÃO

A Teoria da Computação estuda modelos formais capazes de reconhecer linguagens e decidir propriedades sobre cadeias (palavras). Entre esses modelos, os autômatos finitos e as gramáticas regulares são fundamentais por representarem, de forma matemática e operacional, problemas de reconhecimento que aparecem em aplicações práticas como validação de padrões, análise léxica (compiladores), protocolos, filtragem de entradas e modelagem de sistemas reativos.Este projeto implementa um ambiente interativo em Python para definir e manipular modelos de linguagens regulares. O usuário consegue:

- Configurar um Autômato Finito Determinístico (AFD) e um Autômato Finito Não Determinístico (AFND) a partir dos seus componentes formais.
- Validar palavras (cadeias) exibindo o rastreamento do processamento (caminho de estados no AFD e conjunto de estados ativos no AFND).
- Converter um AFND (com transições ε) em um AFD equivalente por meio do ε-fecho e do método de construção de subconjuntos (subset construction).
- Aplicar minimização de AFD, produzindo um autômato equivalente com menos estados, incluindo tratamento de estado morto e remoção de estados inatingíveis.
- Definir uma Gramática Regular Linear à Direita e verificar se uma palavra pertence à linguagem gerada, além de gerar uma árvore de derivação para palavras aceitas (no caso regular linear).
- Como suporte didático, o sistema também exibe a tabela de transição dos autômatos em formato tabular, o que facilita auditoria, depuração e comparação entre as máquinas antes e depois de conversão/minimização.

2. DEFINIÇÕES

Teoria da Computação é a área que estuda modelos formais de computação e os limites do que pode ser reconhecido ou decidido por algoritmos. Dentro dela, linguagens formais e seus mecanismos de descrição/reconhecimento são um eixo central.

Alfabeto (Σ) é um conjunto finito de símbolos. Palavra (w) é uma sequência finita de símbolos de um alfabeto. Linguagem (L) é um conjunto de palavras sobre um alfabeto. O problema de reconhecimento consiste em decidir se uma palavra pertence ou não a uma linguagem.Autômato finito é um modelo matemático de reconhecimento de linguagens

baseado em um conjunto finito de estados e uma função de transição. Um autômato processa uma palavra símbolo a símbolo e, ao final, decide aceitação com base no estado alcançado.

Autômato Finito Determinístico (AFD) é definido pela 5-tupla $M = (\Sigma, Q, \delta, q0, F)$, onde Q é o conjunto finito de estados, q0 é o estado inicial, F é o conjunto de estados de aceitação e δ é a função de transição do tipo $\delta: Q \times \Sigma \rightarrow Q$. A característica determinística é que, para cada par (estado, símbolo), existe uma única computação possível, sem ambiguidades.

Autômato Finito Não Determinístico (AFND) também é representado por $M = (\Sigma, Q, \delta, q0, F)$, porém a função de transição é do tipo $\delta: Q \times \Sigma \rightarrow P(Q)$, permitindo que, para um mesmo par (estado, símbolo), existam múltiplos estados de destino possíveis. O não determinismo significa que o reconhecimento pode ser entendido como a existência de pelo menos um caminho de computação que leva a um estado de aceitação.

Minimização de AFD é o processo de construir um AFD equivalente com o menor número possível de estados. O princípio fundamental é identificar e fundir estados equivalentes, isto é, estados indistinguíveis por qualquer continuação de entrada em relação à aceitação. Técnicas de minimização tipicamente usam particionamento do conjunto de estados em classes de equivalência e refinam essas classes até estabilizar.

Gramática formal é um mecanismo de geração de linguagens baseado em regras de produção. Uma gramática é composta por símbolos não terminais, símbolos terminais, um símbolo inicial e um conjunto de produções que descrevem como cadeias podem ser derivadas.

Gramática regular é uma classe de gramática que gera exatamente as linguagens regulares, sendo equivalente, em poder de expressão, aos autômatos finitos. Em particular, gramáticas lineares à direita possuem produções com restrições estruturais que mantêm a regularidade, permitindo derivação controlada e compatível com reconhecimento por autômatos finitos.

Derivação é a sequência de aplicações de produções a partir do símbolo inicial até obter uma cadeia composta apenas por terminais (ou a cadeia vazia, quando permitida). Esse processo formaliza como uma palavra pertence à linguagem gerada pela gramática.

Árvore de derivação é uma representação hierárquica da derivação, evidenciando a estrutura das aplicações de produções e a relação entre símbolos não terminais expandidos e a cadeia terminal resultante. Ela é uma forma de justificar formalmente a geração de uma palavra por uma gramática.

3. CONSTRUÇÃO DO DOCUMENTO

O conteúdo encontrado neste documento foi construído utilizando os conhecimentos adquiridos nas aulas do professor Otávio, pesquisas no google e com auxílio da Inteligência Artificial da Google popularmente conhecido como Gemini.

Como citado, todo o conteúdo foi montado com os conhecimentos adquiridos nos diferentes tipos de mídias do Professor Otávio Noura Teixeira, ressaltando principalmente as aulas gravadas e disponibilizadas na plataforma "Discord", contamos também com algumas pequenas pesquisas no google para tirar dúvidas simples e com o auxílio do Gemini para impulsionar o início da organização do nosso documento

3.1 Desenvolvimento do Simulador Base (AFD): Pedido de criação de um código em Python capaz de receber dados do usuário para modelar um Autômato Finito Determinístico (AFD) e validar se uma palavra pertence ou não à linguagem.

Chat, gostaria que você utilizasse a linguagem python para criar um código que seja capaz de reconhecer uma máquina automato finito deterministico (todas as informações sobre ele devem ser informadas pelo usuário, inclusive os caminhos que os estados vão tomar) e que também fosse capaz de reconhecer uma palavra

Com certeza! Desenvolver um simulador de **Autômato Finito Determinístico (AFD)** em Python é um excelente exercício de lógica, pois a estrutura do código reflete exatamente a definição matemática do autômato: um conjunto de estados, um alfabeto, uma função de transição, um estado inicial e um conjunto de estados de aceitação.

Abaixo, apresento um código robusto que solicita as configurações ao usuário e, em seguida, valida se uma palavra pertence ou não à linguagem definida.

3.2 Visualização de Dados (Tabela de Transição): Solicitação de uma função, utilizando a biblioteca pandas, para formatar e exibir a função de transição do autômato em formato de tabela acadêmica.

Faça uma função para exibir a função de transição (transições) que a construa como a imagem em anexo, tem liberdade para usar a biblioteca que quiser, mas gostaria que me desse pelo menos uma opção utilizando a pandas

Mostrar código <>

Esta é uma excelente ideia. A visualização tabular (Tabela de Transição) é a forma padrão de representar autômatos em livros e artigos, pois é muito mais legível que uma lista de regras.

Como solicitado, aqui está uma função robusta utilizando **Pandas** que formata a tabela com as convenções acadêmicas:

- → indica o estado inicial.
- * indica estados de aceitação.

3.3 Expansão para Não-Determinismo (AFND): Implementação de lógica avançada com conjuntos (sets) para permitir a criação de Autômatos Finitos Não-Determinísticos (AFND) e a validação de palavras processando múltiplos caminhos simultâneos.

E que tal criar uma função para reconhecer autômatos finitos não determinísticos e validar as suas palavras

Excelente pedido. A principal diferença entre o AFD (Determinístico) e o AFN (Não-Determinístico) é que, no AFN, **um estado pode levar a vários estados diferentes** com o mesmo símbolo de entrada (ou a nenhum).

Para implementar isso em Python, em vez de rastrear um único "estado atual", nós rastreamos um **conjunto de estados atuais**. É como se estivéssemos simulando todos os caminhos possíveis simultaneamente.

Aqui está o código completo, incluindo a entrada de dados adaptada para múltiplas transições e a visualização com Pandas.

3.4 Algoritmo de Conversão (AFND para AFD): Pedido de implementação do algoritmo de Construção de Subconjuntos (*Subset Construction*) para converter automaticamente um AFND em um AFD equivalente.

O próximo passo lógico seria implementar a conversão automática desse AFN para um AFD equivalente (algoritmo de *subset construction*). Você gostaria de ver como isso é feito?

Sim adoraria

Perfeito. A conversão de um Autômato Finito Não-Determinístico (AFN) para um Autômato Finito Determinístico (AFD) é feita através de um algoritmo clássico chamado **Construção de Subconjuntos** (*Subset Construction*).

O Conceito

A ideia principal é: **cada estado do novo AFD será um conjunto de estados do antigo AFN.**

Se no AFN, ao ler '0', você pode ir para `q0` OU `q1`, no AFD criaremos um único estado chamado `{q0, q1}`. O autômato determinístico "simula" estar em todos esses estados ao mesmo tempo.

3.5 Explicação Teórica do Motor AFND: Solicitação de explicação detalhada sobre o funcionamento do código de validação de palavras no AFND, desmembrando a lógica de multiversos e interseção de conjuntos de aceitação.

```
if __name__ == "__main__":
    configurar_e_validar_afn()
```

Oi chat, gostaria que você me explicasse em detalhes a sessão de validação da palavra presente neste código

Com certeza! A sessão de validação de palavras é o "motor" do Autômato Finito Não-Determinístico (AFN). Ela é a parte mais fascinante do código porque precisa simular uma característica fundamental do AFN: **a capacidade de estar em vários estados ao mesmo tempo.**

Enquanto um AFD (Determinístico) segue uma única linha reta (como um jogo de tabuleiro com um único peão), o AFN tem o poder de se "clonar" sempre que há uma bifurcação, explorando múltiplos caminhos simultaneamente.

Vou detalhar linha por linha como esse "multiverso" de estados foi programado utilizando conjuntos (`sets`) em Python.

3.6 Rastreamento e Logs de Execução: Adição de uma funcionalidade para imprimir o passo a passo do processamento da palavra (rastreamento dos estados ativos) no AFND, facilitando a visualização e depuração.

Você notou que no seu código há uma lista chamada `caminho_log` que guarda o histórico por onde o AFN passou, mas ela não está sendo exibida no final? Gostaria que eu ajustasse o código para imprimir a árvore de caminhos percorrida para facilitar a visualização de como a palavra foi processada?

Por favor, eu adoraria

Perfeito! Ver o "passo a passo" do autômato é a melhor forma de entender como o não-determinismo funciona na prática, pois você consegue visualizar exatamente quais estados estão ativos simultaneamente a cada letra lida.

Aqui estão as duas partes do código que foram atualizadas. Você pode simplesmente substituir esses blocos no seu código original.

3.7 Implementação do Epsilon-Fecho: Criação de uma função matemática independente para calcular transições vazias, corrigindo rotas não determinísticas complexas e integrando-a ao algoritmo de conversão.

Prefiro que você faça a função do epsilon-fecho para que eu possa chamá-la em outros momentos além de na função de conversão

Isso é uma excelente prática de programação! Extrair o cálculo do **Epsilon-Fecho** para uma função global deixa o seu código muito mais modular e reutilizável.

Com essa função isolada, você poderá usá-la não apenas na conversão, mas também se quiser fazer a sua função `ValidarPalavra(AFND)` rodar palavras que contenham transições vazias no meio do caminho sem precisar convertê-las para AFD primeiro.

Aqui está a função independente e como o seu código de conversão fica mais limpo ao chamá-la.

3.8 Algoritmo de Minimização de AFD: Solicitação de uma função para minimizar autômatos utilizando o Algoritmo de Particionamento de Moore, incluindo o tratamento para gerar automaticamente "Estados Mortos" (sintéticos) em autômatos parciais.

Ótimo agora gostaria que você criasse uma função def minimização_afd para minimizar autômatos, tomando cuidado com a função de transição dele, caso não seja completa será necessária a criação dos estados sintéticos

A minimização de autômatos é o processo de reduzir o número de estados ao menor possível, removendo estados **inatingíveis** e agrupando estados **equivalentes**.

Como você bem notou, para o algoritmo de minimização (geralmente o de Particionamento de Moore ou o de Hopcroft) funcionar corretamente, o AFD precisa ser **totalmente definido**. Se houver transições faltando (os hífens `-` na sua tabela), precisamos criar um **Estado Morto (Trap State)** para onde todas as transições "vazias" serão direcionadas.

3.9 Depuração Contínua (Debugging): Resolução conjunta de diversos erros (bugs) comportamentais e de sintaxe (como laços for-else acidentais, problemas de indentação em listas e tratamento de KeyError gerado por inconsistências na digitação do usuário).

Levando em consideração o AFD acima e o código que eu mandei junto, ocorreu um erro na minha execução, apontando para o trecho "novas_transicoes[nome_novo][simbolo] = mapa_representante[destino_antigo]" KeyError: 'Q4'

Esse é um excelente caso de depuração! O fato de o erro ter estourado exatamente nessa linha revela um detalhe muito peculiar sobre como os dados chegaram à função de minimização.

O erro `KeyError: 'Q4'` significa que o seu algoritmo tentou procurar o novo nome (M0, M1, etc.) para o estado `'Q4'`, mas o `'Q4'` simplesmente **não existia** no dicionário `mapa_representante`.

Como a lógica do seu Algoritmo de Moore está corretíssima, a única forma de o `Q4` não entrar no `mapa_representante` é se ele foi "jogado fora" na etapa de estados válidos.

3.10 Persistência de Dados (JSON): Pedido para implementar funções de salvamento e carregamento de arquivos .json, permitindo armazenar o progresso e as configurações das máquinas criadas para sessões futuras.

Por favor, crie essa função para salvá-los em json

Implementar a persistência de dados é o toque final que transforma o seu script em uma ferramenta completa. O formato JSON (*JavaScript Object Notation*) é perfeito para isso, pois a estrutura dele é virtualmente idêntica à estrutura de dicionários nativa do Python.

Como o nosso dicionário `maquina_atual` já guarda tudo de forma limpa (strings, listas e dicionários aninhados), nós não precisamos fazer nenhuma conversão complexa. A biblioteca padrão do Python faz todo o trabalho pesado.

Aqui estão as funções de salvar e carregar, junto com a atualização do menu.

3.11 Legenda de Mapeamento na Minimização: Implementação de um rastreio visual na etapa de minimização para exibir ao usuário quais estados antigos foram aglutinados para formar os novos estados minimizados (ex: M0 representa {q2, q3}).

Gemini, assim como você fez para a sessão de conversão de AFND para AFD, em que eram mostrado quais estados antigos (ex:q0) os estados novos (ex: Q0) representam, gostaria que fizesse o mesmo para a minimização

Essa é uma excelente ideia! Mostrar o mapeamento na minimização é tão importante quanto na conversão, pois permite que você rastreie exatamente quais estados o algoritmo considerou "inúteis" ou "redundantes" e agrupou.

Para implementar isso, nós vamos interceptar a etapa de **Reconstrução do AFD minimizado** (lá no finalzinho da função `minimizacao_afd`), que é exatamente o momento em que o código batiza os grupos sobreviventes com os nomes `M0`, `M1`, etc.

Aqui está o trecho atualizado. Você só precisa substituir a parte final da sua função `minimizacao_afd` (logo após o `while True` terminar) por este bloco:

[CHAT DE CONVERSA COM A I.A]

[ACESSO AO CÓDIGO VIA GOOGLE COLAB]

4. CONCLUSÃO

O desenvolvimento deste Simulador Interativo consolidou de forma prática os fundamentos da disciplina de Teoria da Computação I (T.C.I). A modelagem do AFND sobre o alfabeto {a, b, c} e sua subsequente conversão para um AFD, através do método do subconjunto, comprovou empiricamente a equivalência entre representações não determinísticas e determinísticas. A eficácia do algoritmo foi atestada por meio de testes rigorosos de aceitação e rejeição de cadeias, garantindo a integridade do comportamento esperado. Mais do que um exercício de abstração matemática, este projeto consolidou a base lógico-estrutural essencial para o desenvolvimento futuro de analisadores léxicos, compiladores e arquiteturas complexas em linguagens de programação.